# Heart Disease Risk Prediction

End-to-end MLOps: model development, CI/CD, containerisation, Kubernetes deployment and monitoring

**MLOps (AIMLCZG523) - Assignment 01**

Student Name:JAYAKUMAR N | BITS ID:2025AF05062

Code repo - njkumar92fire-star/BITS_Assignment_MLops

Video walkthrough:

# 1. Introduction

The goal is to predict whether a patient has heart disease from routine clinical measurements, and to ship that model the way a production team would: reproducible training, tracked experiments, automated tests, a container image, a Kubernetes deployment and basic monitoring.

**Dataset.** UCI Heart Disease, Cleveland subset: 303 patients, 13 clinical features and a diagnosis label (0 = none, 1-4 = increasing severity). The label is collapsed to binary (disease / no disease), giving 164 negatives and 139 positives (45.9% positive).

**Outcome.** Three models were tuned with 5-fold cross-validation. Logistic regression was selected on cross-validated ROC-AUC (0.904) and scored 0.968 ROC-AUC, 0.885 accuracy and 0.929 recall on a held-out set of 61 patients. It is served through a FastAPI container on Kubernetes with Prometheus and Grafana monitoring.

**A note on the numbers.** The test set is small (61 patients, so one patient is worth about 1.6 accuracy points) and the three models are within noise of each other. The choice of logistic regression rests on its CV score, its simplicity and its interpretability, not on a decisive performance gap.

**Scope and structure.** The project covers the full path from raw data to a running service: data cleaning and exploratory analysis (Sections 3 and 4), model tuning and selection (Section 4), experiment tracking (Section 5), packaging and reproducibility (Section 6), automated testing and CI/CD (Section 7), containerisation (Section 8), Kubernetes deployment (Section 9), and monitoring and logging (Section 10). Section 11 describes the architecture and the design decisions behind it. Section 12 concludes the report, and Section 13 lists the limitations and future work.

# 2. Setup and installation

Python 3.11 or 3.12 is recommended. All versions are pinned in requirements.txt (training) and requirements-serving.txt (container).

```
git clone https://github.com/njkumar92fire-star/BITS_Assignment_MLops && cd
BITS_Assignment_MLops
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
python -m src.download_data          # download from UCI and write
data/processed/heart_clean.csv
python -m src.train                  # tune, evaluate, log to MLflow, save
models/model.joblib
pytest -v                            # 21 unit tests
mlflow ui --backend-store-uri sqlite:///mlflow.db     # http://localhost:5000
docker build -t heart-api:1.0.0 . && docker run -p 8000:8000 heart-api:1.0.0
```

The repository layout is: src/ (data, features, training, inference), api/ (FastAPI service), tests/, notebooks/ (EDA, training, inference), k8s/ (Deployment, Service, Ingress, HPA), monitoring/ (Prometheus and Grafana configuration), .github/workflows/ci.yml, Dockerfile, docker-compose.yml, screenshots/ and reports/.

# 3. Data acquisition and EDA

## 3.1 Acquisition and cleaning

src/download_data.py fetches processed.cleveland.data from the UCI repository (with a fallback to a local zip or the committed copy if the network is down) and writes a cleaned CSV. Cleaning consists of: converting the '?' missing-value marker to NaN, collapsing the 0-4 label to a binary target, removing duplicate rows (none were found) and casting categorical codes to numeric.

Six values are missing (4 in ca, 2 in thal, about 2% of rows). They are deliberately not filled during cleaning. Imputation sits inside the scikit-learn Pipeline so the fill values are learned from training folds only, which avoids leaking information from validation or test rows.

## 3.2 Class balance

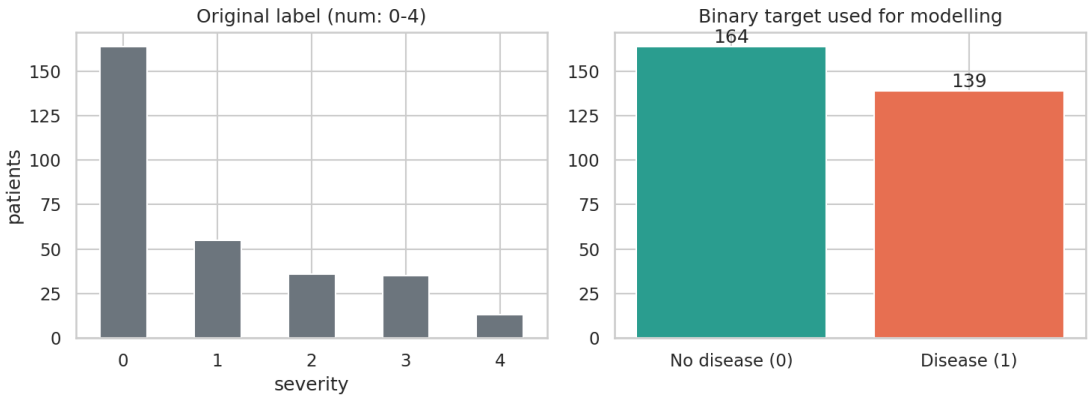



*Figure 1. Original severity label (left) and the binary target used for modelling (right).*

The classes are close to balanced (54% / 46%), so accuracy is informative and no resampling is used. Recall and ROC-AUC are still reported because a missed diagnosis is costlier than a false alarm.

## 3.3 Numeric features

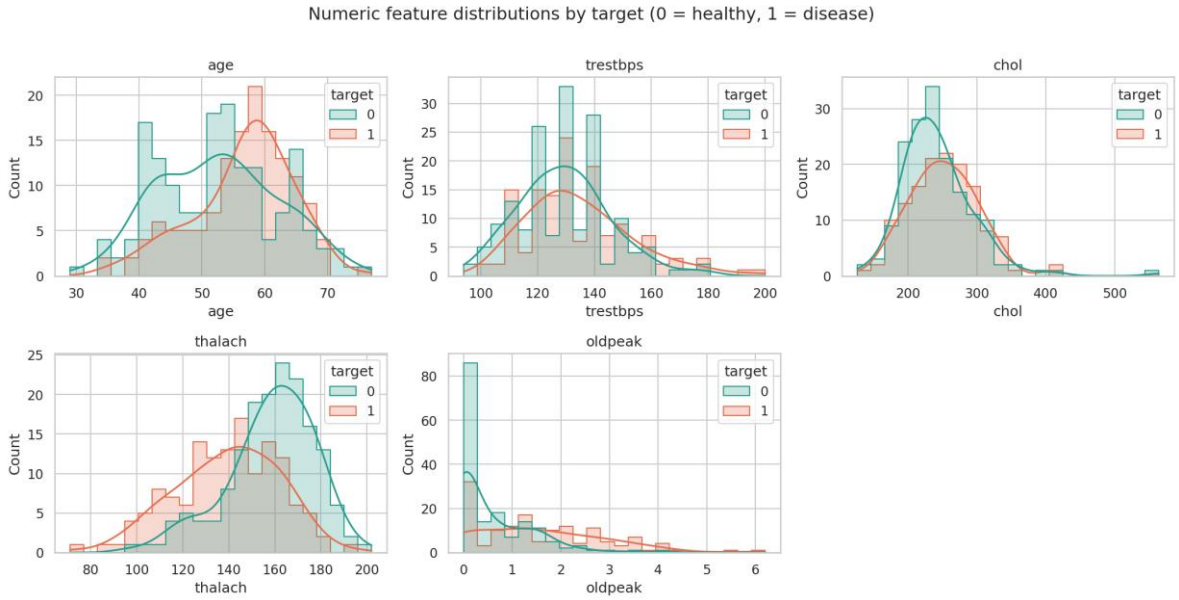



*Figure 2. Distributions of the numeric features split by target.*

Patients with disease are older on average (56.6 vs 52.6 years), reach a lower peak heart rate (139 vs 158 bpm) and show larger exercise-induced ST depression (oldpeak 1.6 vs 0.6). Cholesterol and resting blood pressure overlap heavily between the groups. oldpeak is right-skewed and chol has a few extreme values (maximum 564 mg/dl), which is one reason numeric columns are standardised.

## 3.4 Categorical features

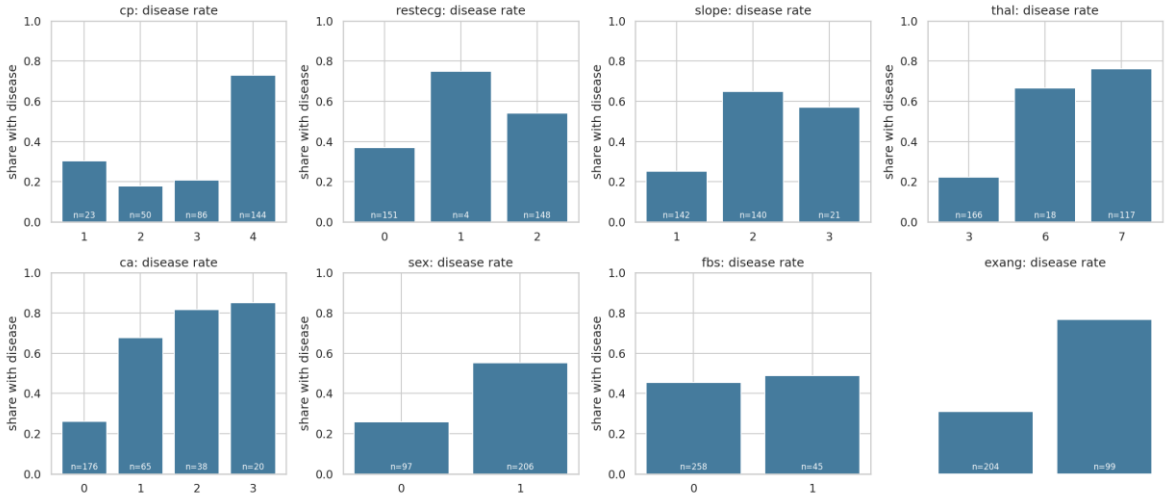



*Figure 3. Share of patients with disease within each category (n shown on bars).*

The strongest separators are chest pain type (asymptomatic, cp = 4, has a 73% disease rate against 18-30% for the other types), exercise angina (77% vs 31%), number of coloured vessels (26% at 0, rising to 85% at 3) and thalassemia result (76% for reversible defect vs 22% for normal).

## 3.5 Correlations

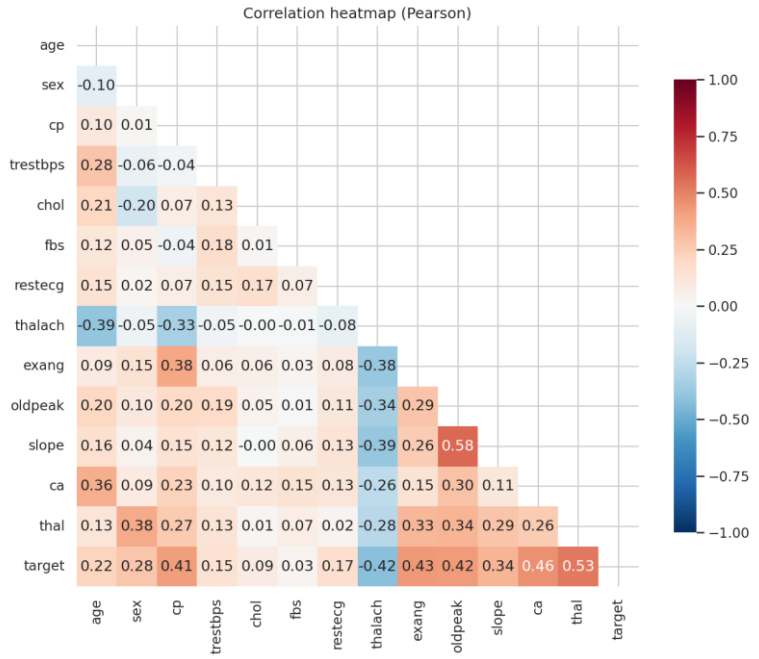



*Figure 4. Pearson correlation heatmap.*

The most correlated features with the target are thal (0.53), ca (0.46), exang (0.43), oldpeak (0.43), thalach (-0.42) and cp (0.41). The largest correlation between any two predictors is 0.58, so multicollinearity is not a concern for the linear model. chol and fbs have almost no linear correlation

with the target (0.09 and 0.03) but were kept. Pearson correlation on ordinal and categorical codes is only a rough guide, so the disease-rate bars in Figure 3 are the more reliable evidence for those columns.

# 4. Feature engineering and model development

## 4.1 Preprocessing pipeline

| Feature group | Columns | Steps |
|---|---|---|
| Numeric | age, trestbps, chol, thalach, oldpeak, hr_ratio | median imputation, standardisation |
| Categorical | cp, restecg, slope, thal, ca | most-frequent imputation, one-hot (unknown categories ignored) |
| Binary | sex, fbs, exang | most-frequent imputation, passthrough |

One engineered feature was added: hr_ratio = thalach / (220 - age), the fraction of the age-predicted maximum heart rate the patient reached. The motivation is that older patients naturally peak lower, so raw thalach mixes age and fitness. An ablation (repeated 5x5 CV, logistic regression) showed no measurable gain: ROC-AUC was 0.8928 both with the feature and without it, so it adds no measurable gain. hr_ratio is nearly a linear combination of age and thalach, which the model already has. It stays in the pipeline as a documented domain feature, but it is not claimed to improve performance.

The whole preprocessing chain is part of the fitted Pipeline object, so training and serving apply identical transformations. A custom function is used for the feature step (not a lambda) so the pipeline can be pickled and reloaded in the API container.

## 4.2 Candidate models and tuning

A stratified 80/20 split (242 train / 61 test, random_state = 42) is made before any fitting. Each model is tuned with GridSearchCV (stratified 5-fold, scoring = ROC-AUC) on the training portion only.

| Model | Grid searched | Selected parameters |
|---|---|---|
| Logistic Regression | C in {0.01, 0.1, 1, 10} | C = 1 |
| Random Forest | n_estimators {100, 300}, max_depth {3, 5, None}, min_samples_leaf {1, 3} | 300 trees, max_depth 3, min_samples_leaf 3 |
| Gradient Boosting | n_estimators {100, 200}, learning_rate {0.05, 0.1}, max_depth {2, 3} | 100 trees, learning_rate 0.05, max_depth 2 |

## 4.3 Cross-validated results (training split, 5-fold)

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.847 ± 0.010 | 0.877 ± 0.054 | 0.783 ± 0.046 | 0.825 ± 0.007 | 0.904 ± 0.014 |
| Random Forest | 0.826 ± 0.034 | 0.860 ± 0.073 | 0.755 ± 0.095 | 0.797 ± 0.044 | 0.896 ± 0.025 |
| Gradient Boosting | 0.822 ± 0.026 | 0.829 ± 0.064 | 0.783 ± 0.068 | 0.801 ± 0.030 | 0.878 ± 0.028 |

*Table 1. Mean ± standard deviation across folds.*

## 4.4 Held-out test results

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.885 | 0.839 | 0.929 | 0.881 | 0.968 |
| Random Forest | 0.852 | 0.828 | 0.857 | 0.842 | 0.949 |
| Gradient Boosting | 0.902 | 0.867 | 0.929 | 0.897 | 0.959 |

*Table 2. Performance on the 61 held-out patients.*

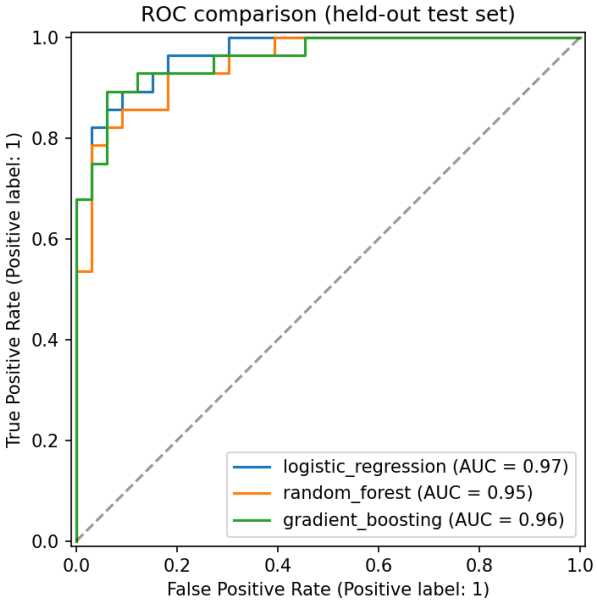



*Figure 5. ROC curves of the three tuned models on the held-out set.*

## 4.5 Model selection and how to read the results

- Selection used cross-validated ROC-AUC only. The test set played no part in choosing the model. Logistic regression had the highest CV ROC-AUC, but the gap to random forest (0.904 vs 0.896) is smaller than one fold standard deviation.
- The CV scores of a tuned model are slightly optimistic because the same folds chose the hyper-parameters. The held-out set is the cleaner estimate, but with 61 patients it is noisy.
- Every model scored higher on the test set than in CV (for logistic regression 0.968 vs 0.904 ROC-AUC). This points to a comparatively easy split rather than a better model, so the CV figures are the more conservative expectation of real-world performance.
- Gradient boosting has the best test accuracy (0.902) but the lowest CV ROC-AUC (0.878). On this dataset size the ranking between models is not reliable.

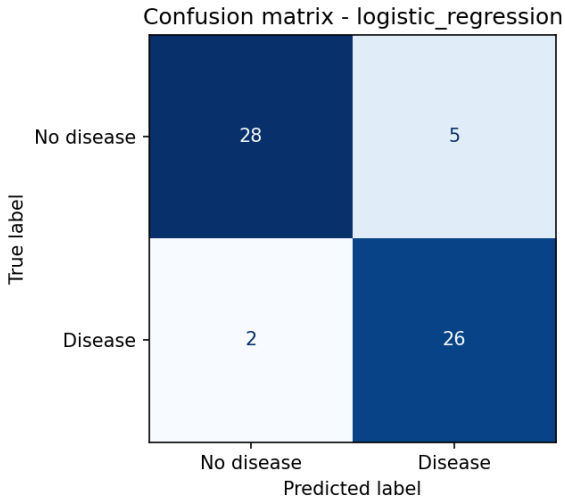



*Figure 6. Confusion matrix of the selected model on the test set: 28 true negatives, 5 false positives, 2 false negatives, 26 true positives.*

The selected model misses 2 of 28 diseased patients (recall 0.929) at the cost of 5 false alarms. For a screening use case this trade-off is reasonable, and the decision threshold of 0.5 could be lowered further if missing a case is judged more costly. The largest logistic regression coefficients are no vessels coloured (ca = 0, protective), male sex, asymptomatic chest pain (cp = 4) and a reversible thalassemia defect (thal = 7), which agrees with the EDA.

## 5. Experiment tracking (MLflow)

src/train.py logs to MLflow with a SQLite backend (mlflow.db) and local artifact storage (mlartifacts/). One parent run (model-comparison) holds the shared parameters and the dataset, and one nested child run exists per model.

| Logged item | Details |
|---|---|
| Parameters | model type, tuned hyper-parameters, split size, CV folds, random seed, library version, dataset size |
| Metrics | CV mean and std for accuracy, precision, recall, F1, ROC-AUC; grid-search best score; the same five metrics on the test set |
| Artifacts | ROC curve, confusion matrix and feature-importance plot per model; ROC comparison plot; cleaned dataset; metadata.json |
| | each tuned pipeline with signature and input example; the selected model registered as heart-disease-classifier |

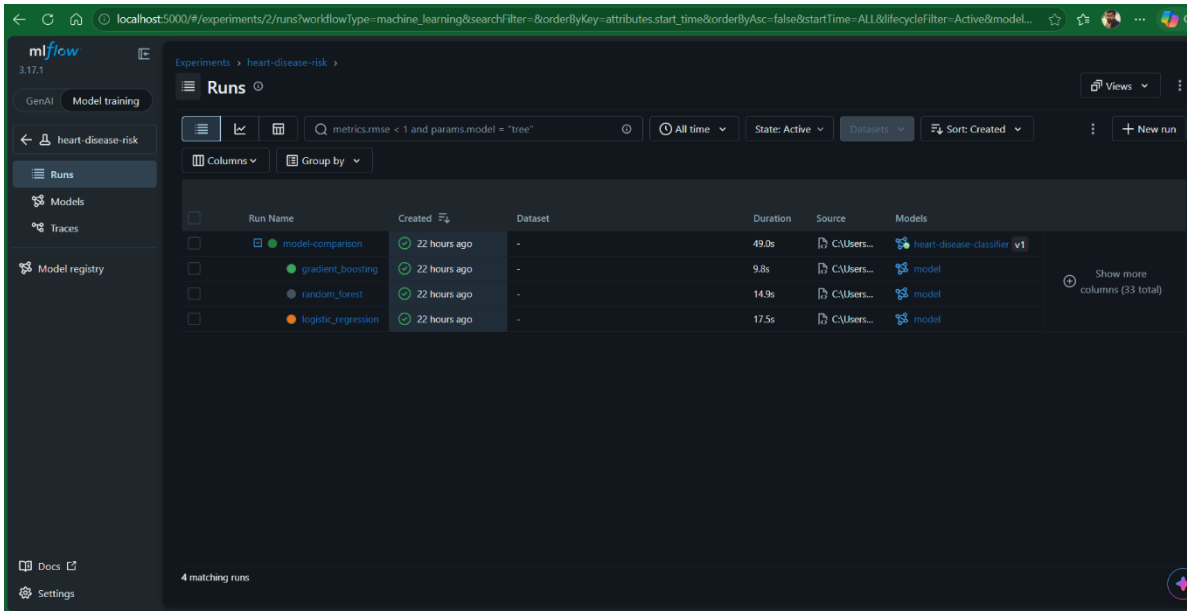



*Figure 7. Experiment list.*

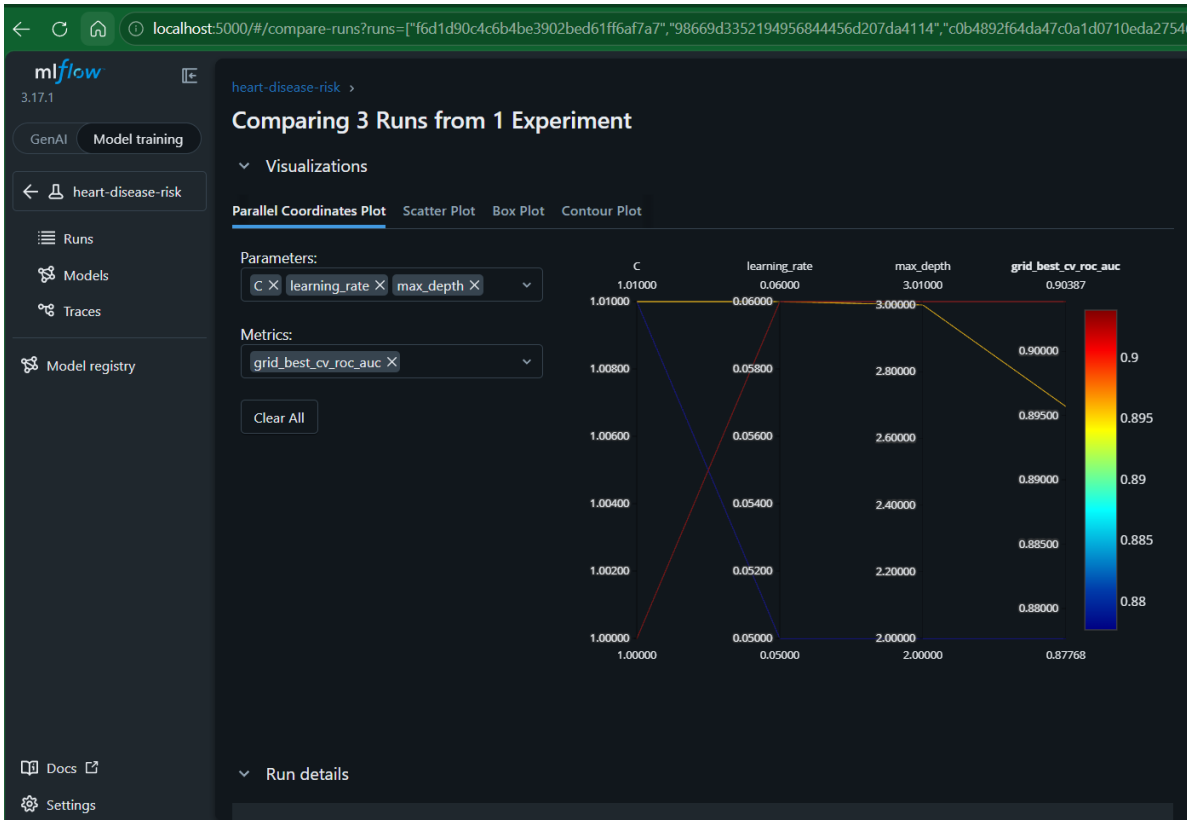



*Figure 8. Run comparison.*

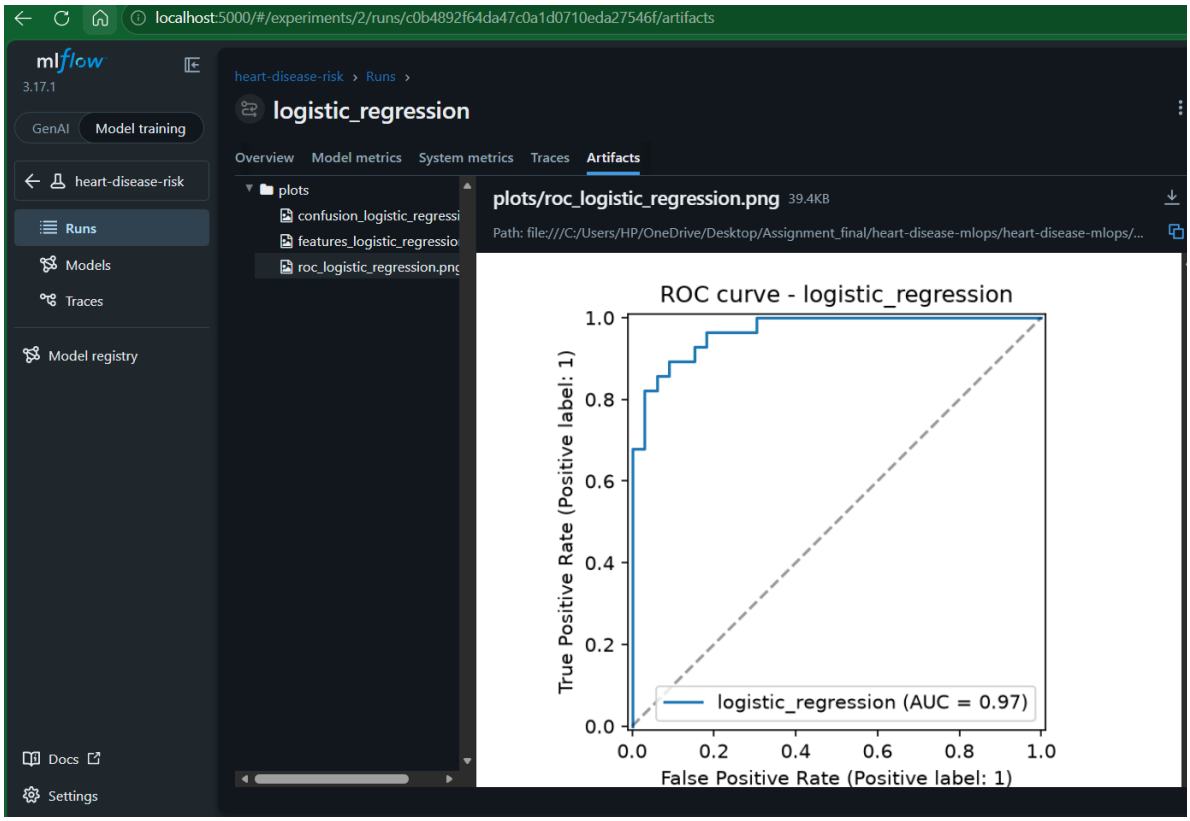



*Figure 9. Artifacts produced by the selected run.*

## 6. Model packaging and reproducibility

- The final artefact is a single scikit-learn Pipeline (preprocessing + classifier) saved with joblib to models/model.joblib, plus models/metadata.json (model name, parameters, metrics, library versions, MLflow run id). The same pipeline is also logged to MLflow as a registered model.
- Because the preprocessing is inside the pipeline, serving code passes raw patient fields and needs no separate transformer files.
- requirements.txt pins exact versions used for training; requirements-serving.txt lists the smaller runtime set used in the container. Pickled models must be loaded with the same scikit-learn version, which is why the versions match and the version is recorded in metadata.json.
- Random seeds are fixed (split, CV, models). Re-running python -m src.train reproduces the reported metrics.
- Security note: joblib and cloudpickle files can execute code when loaded, so model files should only be loaded from trusted sources, as is the case here (built by this repository's own pipeline).

## 7. CI/CD pipeline and automated testing

21 unit tests (pytest) cover the data cleaning, the feature transformer, model training for every candidate, the pickled-model round trip, and the API. Tests use synthetic data in the real schema, so they run without downloading anything.

| Test file | What it checks |
|---|---|
| test_data.py | binary target creation, duplicate removal, missing values preserved for the pipeline, '?' parsing |

| Test file | What it checks |
|---|---|
| test_features.py | hr_ratio value, no input mutation, no NaN after preprocessing, scaling, unseen categories |
| test_model.py | each candidate trains and returns valid probabilities, model save/load round trip, prediction schema, missing optional fields |
| test_api.py | /health, /predict output and request id, validation errors (422), optional fields, /metrics counters |

The GitHub Actions workflow (.github/workflows/ci.yml) has four dependent jobs, so a failure in any stage stops the pipeline and the log shows which step failed:

- **lint:** flake8 over the repository.
- **test:** pytest with coverage; uploads the JUnit and coverage reports as artifacts.
- **train:** checks the download script, runs python -m src.train, uploads model, metadata, MLflow database and plots.
- **docker:** downloads the freshly trained model, builds the image, runs it and smoke-tests /health, /predict and /metrics with curl; uploads the container log.

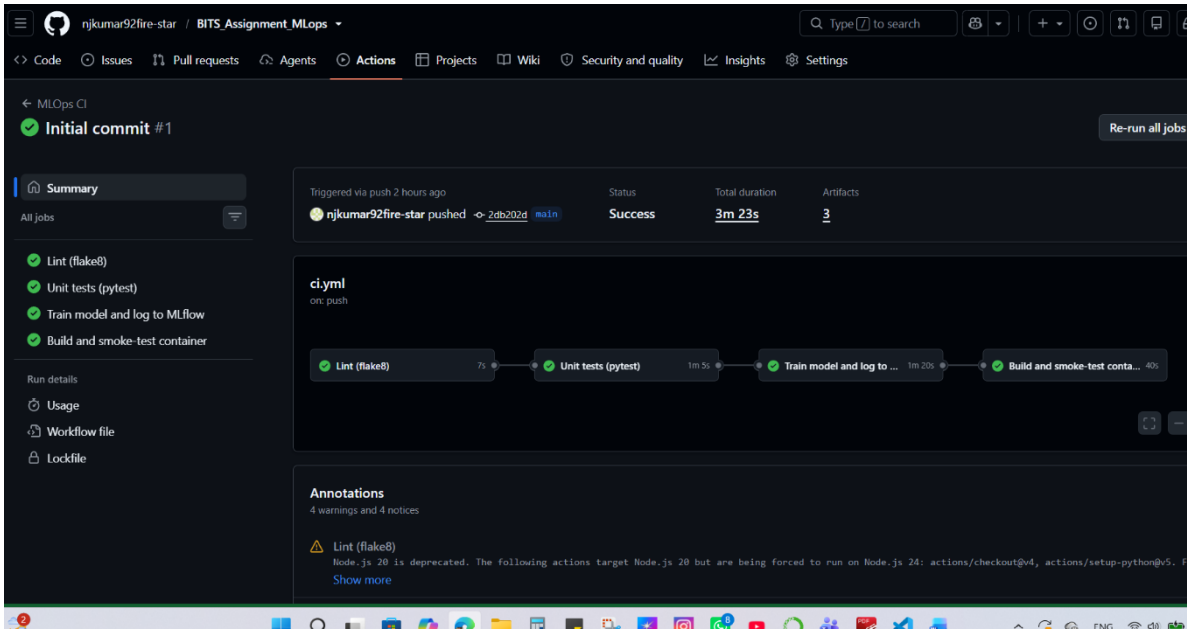



*Figure 10. Successful pipeline run.*

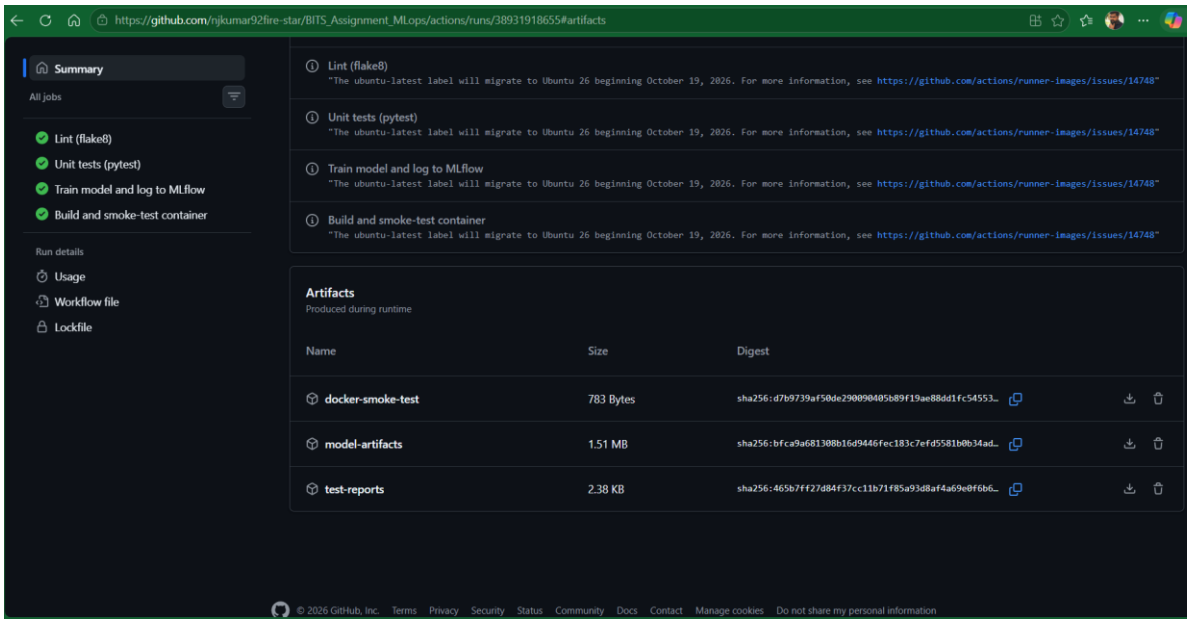



*Figure 11. Artifacts produced by a run.*

## 8. Containerisation

The API is a FastAPI application served by uvicorn, built from python:3.11-slim. The image contains only runtime dependencies, runs as a non-root user and has a HEALTHCHECK.

| Endpoint | Purpose |
|---|---|
| POST /predict | JSON patient record in; prediction, label, confidence, probability of disease, model name and request id out. Inputs are validated (types and clinical ranges); ca and thal are optional and imputed by the pipeline. |
| GET /health | readiness; returns 503 if the model failed to load |
| GET /info | model name, training time, library version and test metrics |
| GET /metrics | Prometheus metrics |

Example request and response:

```
curl -X POST http://localhost:8000/predict -H 'Content-Type: application/json' \
  -d
'{"age":67,"sex":1,"cp":4,"trestbps":160,"chol":286,"fbs":0,"restecg":2,"thalach":108,"exan
g":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":3}'
{"prediction":1,"label":"heart disease
likely","confidence":0.9835,"probability_disease":0.9835,...}
```

On Windows PowerShell, curl is an alias for a different command, so run the example with curl.exe instead, or use Invoke-RestMethod -Uri http://localhost:8000/predict -Method Post -ContentType "application/json" -Body '{"age":67,"sex":1,"cp":4,"trestbps":160,"chol":286,"fbs":0,"restecg":2,"thalach":108,"exang":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":3}', which returns the same prediction.

'confidence' is the model's probability for the predicted class. It is a model probability, not a calibrated clinical risk, and this service is a course exercise, not a medical device.

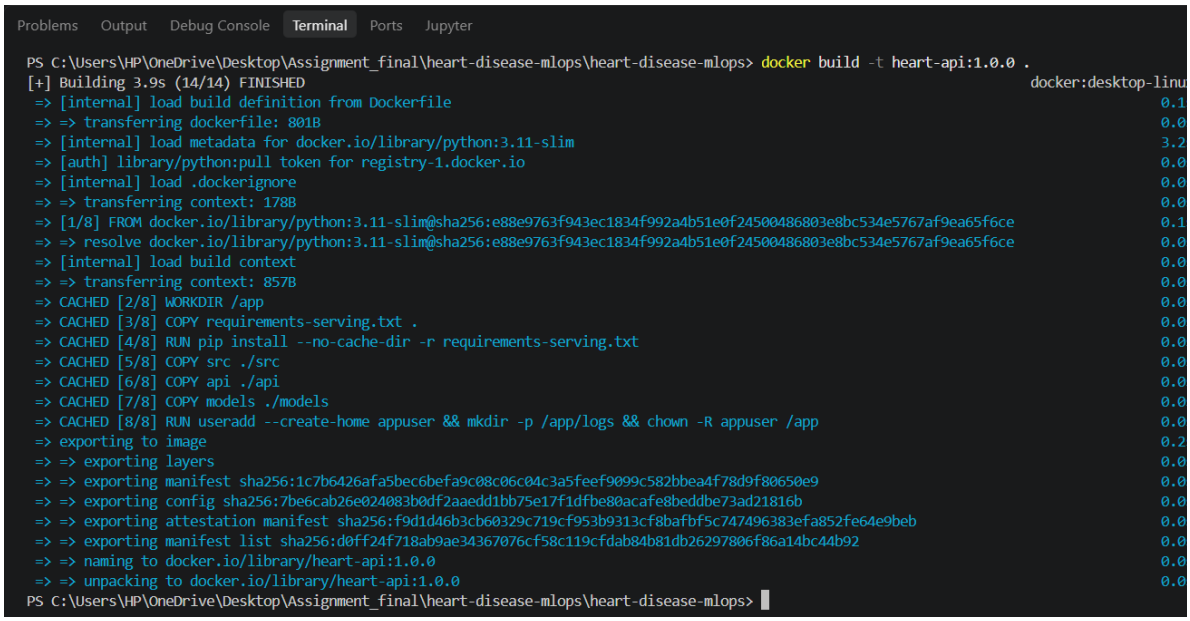



*Figure 12. Image build.*

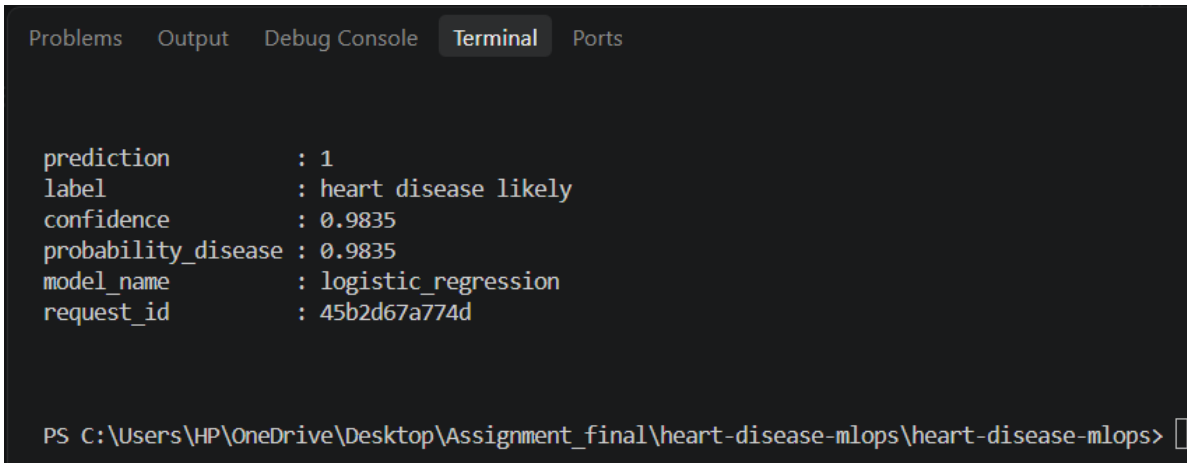



*Figure 13. Container serving a prediction.*

## 9. Production deployment (Kubernetes)

The image is deployed to a local Kubernetes cluster (Minikube or Docker Desktop). The manifests are in k8s/:

- **deployment.yaml:** 2 replicas, rolling update with zero unavailable pods, CPU/memory requests and limits, readiness and liveness probes on /health.
- **service.yaml:** LoadBalancer on port 80 (Minikube: minikube tunnel; Docker Desktop: localhost).
- **ingress.yaml:** optional NGINX ingress alternative.
- **hpa.yaml:** optional autoscaler (2 to 5 replicas at 70% CPU; needs metrics-server).

Figures 14 and 15 were captured on Docker Desktop's built-in Kubernetes, where the LoadBalancer service is reached at localhost. The Minikube commands below are the alternative route.

```
minikube start && eval $(minikube docker-env)      # build the image inside the cluster
docker build -t heart-api:1.0.0 .   &&   kubectl apply -f k8s/deployment.yaml -f
k8s/service.yaml
kubectl get pods,svc            # then: minikube tunnel   and   curl
http://127.0.0.1/health
```

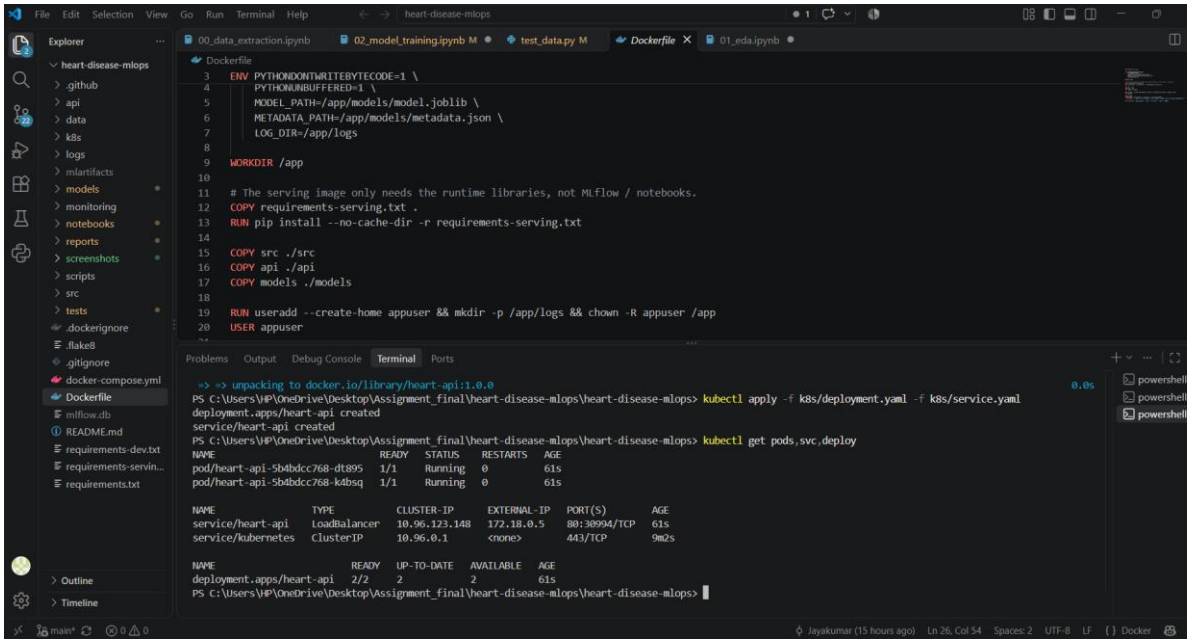



*Figure 14. Cluster state.*

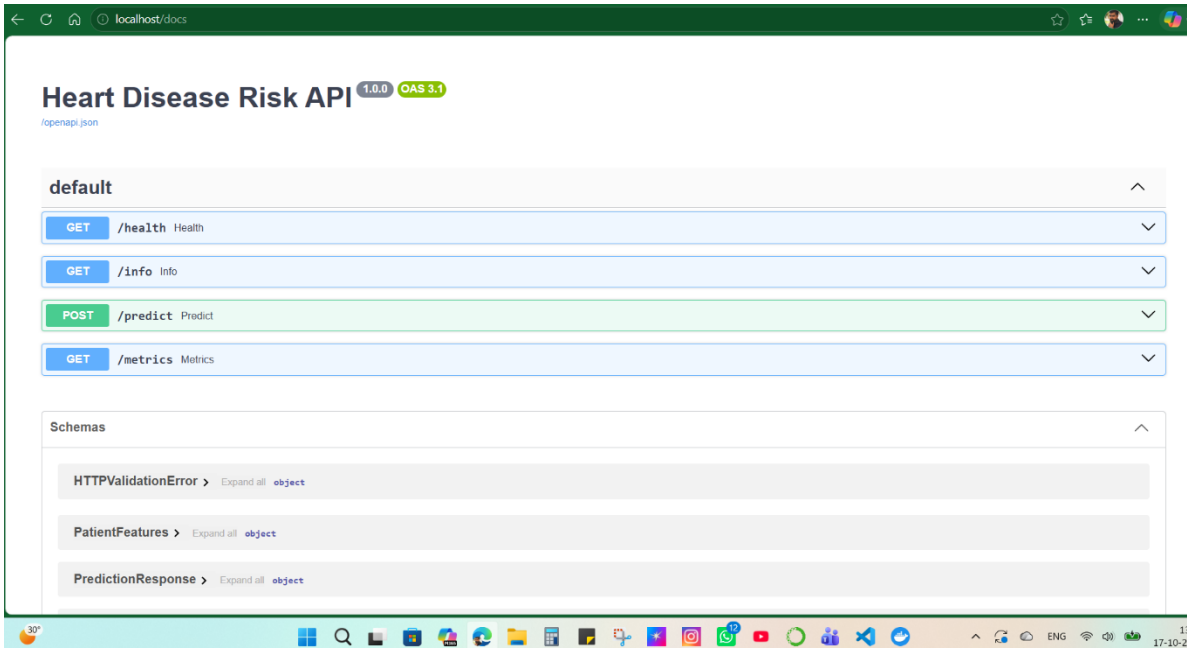



*Figure 15. Endpoint verification through the service.*

# 10. Monitoring and logging

## 10.1 Logging

Every request produces one JSON log line (request id, method, path, status, latency in ms) to stdout and to a rotating file logs/api.log; every prediction adds a line with the predicted class and confidence. The request id is also returned in the X-Request-ID header and in the response body, so a user report can be traced to a log line. Patient feature values are deliberately not logged.

```
{"event": "request", "request_id": "e41abdaf8fdc", "method": "POST", "path": "/predict",
"status": 200, "latency_ms": 3.1}
```

## 10.2 Metrics and dashboard

The API exposes Prometheus metrics: request count by endpoint and status, request latency histogram, predictions by class, histogram of prediction confidence, error count and a model-loaded gauge. Prometheus scrapes /metrics and a provisioned Grafana dashboard (Heart Disease API) shows request rate, p95 latency, predictions by class, total predictions, 5xx errors, median confidence and model status. The prediction-class and confidence panels are a simple input-drift signal: a sudden shift in either would be a reason to investigate the incoming data.

```
docker compose up --build        # API :8000, Prometheus :9090, Grafana :3000 (admin /
admin)
```

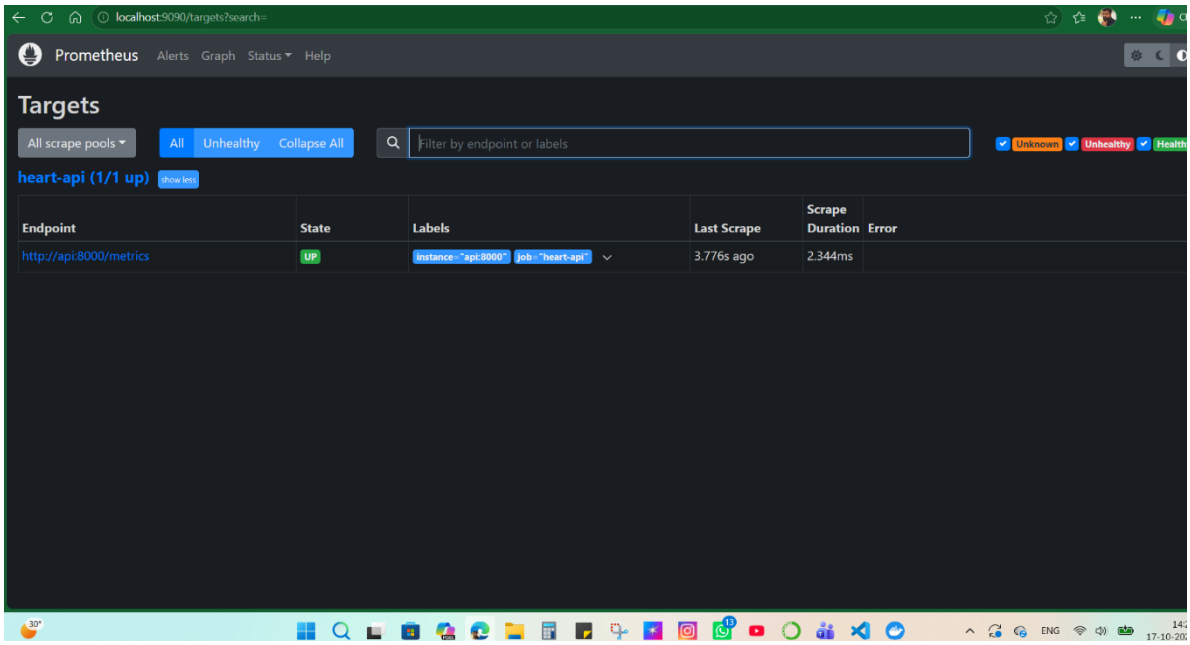



*Figure 16. Prometheus scrape target.*

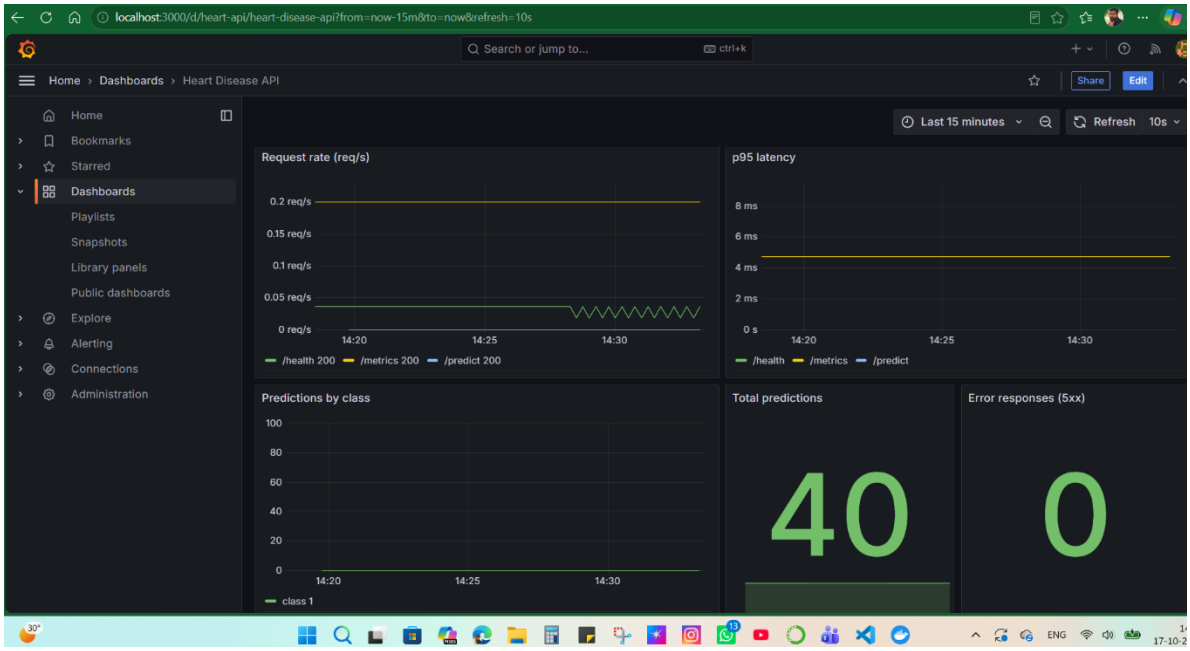



*Figure 17. Monitoring dashboard.*

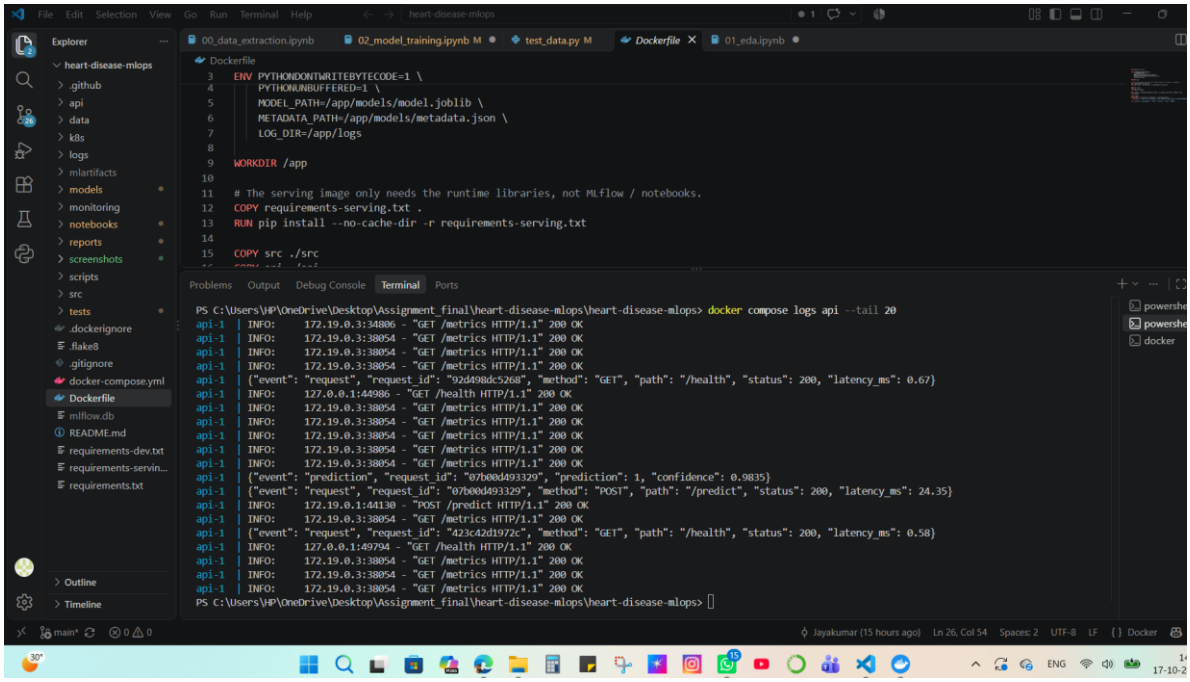



*Figure 18. Request logs.*

## 11. Architecture

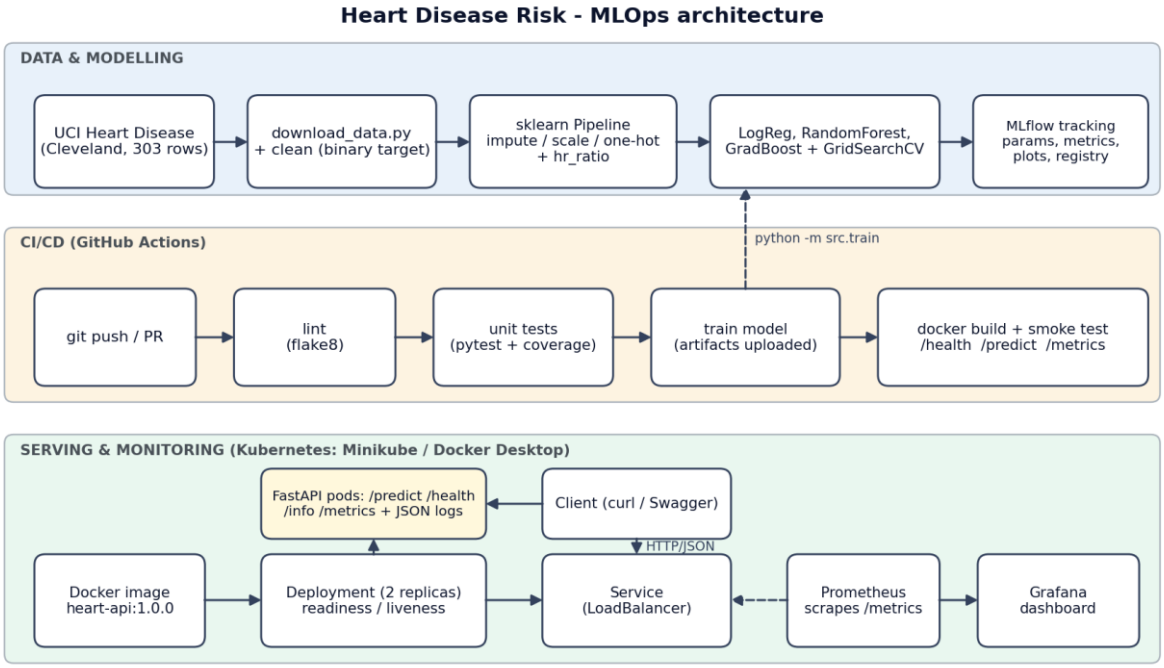



*Figure 19. System architecture: modelling, CI/CD, serving and monitoring.*

Design decisions: (1) preprocessing and model live in one pickled Pipeline to eliminate training/serving skew; (2) the serving image is separate from the training environment and excludes MLflow and notebook dependencies; (3) CI trains the model and then builds the image from that exact artefact, so what is tested is what ships; (4) monitoring is part of the service (metrics endpoint and structured logs) rather than an add-on.

## 12. Conclusion

This project delivered an end-to-end MLOps pipeline for heart disease risk prediction, from reproducible training and tracked experiments through automated CI/CD, containerised serving, Kubernetes deployment and monitoring. Logistic regression was selected on cross-validated ROC-AUC (0.904) and reached 0.968 ROC-AUC and 0.929 recall on the 61-patient held-out set. Given the small test set and the closeness of the three models, these numbers demonstrate the pipeline rather than a clinical claim. The main value lies in the engineering: preprocessing and model travel together as one artefact, the CI pipeline tests what it ships, and monitoring is part of the service itself.

## 13. Limitations and future work

- Small dataset (303 patients from a single clinic). Results will not necessarily generalise to other populations; the dataset is for education, not clinical use.
- The model comparison is statistically weak (see 4.5). Repeated or nested cross-validation would give a firmer ranking and a less optimistic estimate.
- The model file is committed to the repository for simplicity. A larger project would store it in a model registry or object store and deploy by version.
- No authentication or rate limiting on the API, and Grafana uses default credentials; both are acceptable for a local demo only.
- Drift monitoring is limited to prediction-class and confidence distributions. Feature-level drift tests (for example PSI on age and thalach) and a retraining trigger are natural next steps.
- Deployment is to a local cluster; moving to GKE, EKS or AKS would need an image registry, a cloud load balancer and secrets management.